

DB Quest AI

How the Whole App Works — a Plain-English Guide

Version 0.2 — with team logins, roles, a real database, and per-team knowledge. Explains every file, how the pieces talk, and how to plug in an AI API key.

Deutsche Bank Hackathon Project • AI Digital Colleague + AI Escape Missions

1. What is DB Quest AI?

DB Quest AI is a web app for a Deutsche Bank hackathon. It is now a **real multi-team product**: people sign in, join a team, and everything they see belongs to that team.

It does two things in one place:

- **AI Digital Colleague** — answers work questions using your team's own documents (and shows the sources), plus onboarding plans, an acronym explainer, an expert finder, and a document summariser.
- **AI Escape Missions** — compliance training as a game: you make decisions, an AI "Game Master" coaches you, and you get a private score.

Two big promises:

- **Works with zero setup.** With no configuration it runs fully offline — it auto-creates a database, seeds a demo team, and uses a built-in "mock" AI. No keys, no cloud.
- **Upgrades cleanly.** Add an AI key and the same screens quietly switch to a real model and smarter search. Nothing else changes.

New in this version: real **login** (email + password), **teams** (your data is private to your team), **roles** (Admin vs Member), and a **database** so nothing is lost when you restart.

2. The Big Picture

Two programs run at once and talk over your computer's own web address:

- **Frontend** (what you see) — React + Vite + Tailwind at **http://localhost:5173**. Draws pages and never holds any secret answers.
- **Backend** (the brain) — Python + FastAPI at **http://localhost:8000**. Holds the database, checks logins, enforces team rules, and talks to the AI.

The frontend sends small web messages ("API calls") that start with **/api/...** Every call (except health and login) carries two things: a **token** that proves who you are, and a **team id** that says which team's data you want.

```
Browser (React)
| every request adds two headers:
|   Authorization: Bearer <your login token>
|   X-Team-Id: <the team you picked>
|
v
FRONTEND :5173  --(Vite forwards /api)-->  BACKEND :8000 (FastAPI)
                                     |
                                     | check token -> check team membership -> do the work
                                     |
                                     | AIService (mock or live) . retrieval (search) . database (SQLite)
                                     |
                                     v
                                     answer travels back and appears on screen
```

3. The Usage Model & Roles

The whole product follows one simple flow:

```
Sign in (or 'Continue as demo') -> land in a TEAM with a ROLE
Admin uploads documents -> Members ask questions (answers cite those docs)
-> everyone plays missions (scores stay private)
-> Admin views anonymized Team Insights
```

Two roles decide what you can do:

Role	Can do
Admin	Upload documents, create missions, view aggregated team insights (plus everything a member can do).
Member	Ask questions, generate onboarding plans, look up acronyms/experts, play missions.

Admin-only buttons are hidden from members in the UI, and the server blocks them too (a member calling an admin route gets rejected). Every team is fully separated: you can only ever see your own team's documents, missions, and results.

Demo accounts (seeded automatically, password **demo123**):

- demo-admin@dbquest.ai → Admin
- demo-member@dbquest.ai → Member

The **Continue as demo user** button signs you in as the demo admin.

4. A Real Example, Step by Step

You sign in and ask the Digital Colleague: "How do I onboard a new developer?"

- **Step 1 — Sign in.** You click 'Continue as demo user'. The backend checks the account and returns a **token** plus your **teams**. The app remembers them.
- **Step 2 — Ask.** On the Ask page you type the question. The frontend sends a POST to `/api/colleague/ask` with your token and team id attached.
- **Step 3 — Security check.** The backend confirms your token is valid and that you belong to that team.
- **Step 4 — Search YOUR team's docs.** It searches only your team's document chunks for the most relevant paragraphs.
- **Step 5 — Answer.** Those paragraphs go to the AI service (mock or live) which writes the answer; the sources still come from the search, so citations stay honest.
- **Step 6 — Show it.** The frontend shows the answer, clickable source cards, and a confidence badge. If nothing matched, it clearly says 'not found in your team's documents'.

Safety idea kept from before: for missions, the browser is NEVER told which choice is correct. When you pick an option the server grades it — so no one can peek at the answer key.

5. How Login & Team Isolation Work

Passwords are never stored as plain text — they are **hashed** (scrambled one-way) before saving. When you log in, the server hands you a **JWT token**: a signed pass that proves who you are without the server having to remember a session.

Every protected request goes through two gates in **security.py**:

- **get_current_user** — reads the token, checks the signature, and loads your user record. No/!bad token = 401 'Not authenticated'.
- **get_team_context** — reads the **X-Team-Id** header and confirms you are actually a member of that team (else 403). Admin-only routes add **require_admin**.

Because every content query filters by **team_id**, one team can never see another team's data — even if someone tries a different team id by hand (they get 403).

6. The Database (nothing is lost on restart)

Data now lives in a real database using **SQLModel**. By default it's a single **SQLite** file at **backend/storage/dbquest.db**, created automatically on first run. To scale up, point **DATABASE_URL** at Postgres — no code changes.

The tables and how they relate:

Table	What it stores
User	An account: email + hashed password + name.
Team	A workspace / team.
Membership	Links a User to a Team WITH a role (admin or member).
Document	A team's document (full text is saved, so uploads survive restarts).
Chunk	A paragraph of a document, with an optional embedding vector for search.
Mission	A team's mission (the full JSON, including the hidden answer key).
MissionAttempt	One private run of a mission: score, grade, decisions. Feeds insights.

Relationship in words: a **User** joins one or more **Teams** through a **Membership** (which carries the role). Each **Team** owns its **Documents** (split into **Chunks**), its **Missions**, and its **MissionAttempts**.

7. How Document Search Works (per team)

When an admin adds a document, the backend splits it into paragraph-sized **chunks** and stores them for that team. Answering a question then finds the best-matching chunks — in one of two ways:

- **Offline (default): TF-IDF keyword search.** A dependency-free ranking that needs no internet or key. Always available.
- **Live (when a key is set): embeddings.** Each chunk gets a numeric 'meaning' vector; your question is turned into a vector too, and the closest chunks win. This understands meaning, not just exact words.

The search is always restricted to the current team's chunks, so the Colleague can never answer using another team's documents. If embeddings ever fail, it automatically falls back to TF-IDF.

8. The Two Brains: Mock vs Live AI

Every AI feature returns the **same shape** of data whether it came from the offline mock or a real model, so the pages never care which answered.

Capability	Demo mode (default)	Live mode (key set)
------------	---------------------	---------------------

Q&A answers	Mock synthesis	Real LLM, grounded in team docs
Mission generation	Built-in archetypes	Real LLM (forced into schema)
Document search	TF-IDF (offline)	Embeddings (OpenAI/Azure)
Grading / hints / score	Always rule-based	Always rule-based (fair & consistent)

The live path always falls back to the mock on any error, so the app can never fully break during a demo.

9. How to Add Your Own AI API Key

By default the app is in **mock** mode. To use a real AI model you edit ONE file: **.env** inside the **backend** folder.

Step 1 — Create the .env file

In the **backend** folder there is a template **.env.example**. Copy it to **.env**.

```
# Windows PowerShell, inside the backend folder
Copy-Item .env.example .env
```

Step 2 — Choose a provider and paste your key

Open **.env**, change **AI_PROVIDER** from mock to your provider, and paste the key. Pick ONE:

Option A — OpenAI (also enables smart embedding search)

```
AI_PROVIDER=openai
OPENAI_API_KEY=sk-your-key-here
OPENAI_MODEL=gpt-4o-mini
OPENAI_EMBEDDING_MODEL=text-embedding-3-small
```

Option B — Azure OpenAI

```
AI_PROVIDER=azure
AZURE_API_KEY=your-azure-key
AZURE_ENDPOINT=https://YOUR-resource.openai.azure.com
AZURE_DEPLOYMENT=your-chat-deployment
AZURE_EMBEDDING_DEPLOYMENT=your-embedding-deployment
AZURE_API_VERSION=2024-02-01
```

Option C — Google Gemini (chat only; search stays on TF-IDF)

```
AI_PROVIDER=gemini
GEMINI_API_KEY=your-gemini-key
GEMINI_MODEL=gemini-2.0-flash
```

Step 3 — (Optional) set a login secret & database

For real use, also set a strong **JWT_SECRET** (signs login tokens) and, if you want Postgres instead of the built-in SQLite file, a **DATABASE_URL**.

```
JWT_SECRET=a-long-random-string
# Optional Postgres instead of SQLite:
# DATABASE_URL=postgresql+psycopg://user:pass@host:5432/dbquest
```

Step 4 — Restart the backend, then check

The key is read only at startup. Stop the backend (Ctrl+C) and start it again, then open **http://localhost:8000/api/health**. You should see **liveAi: true** (and **embeddings: true** for OpenAI/Azure).

```
{ "status": "ok", "aiProvider": "openai", "liveAi": true,
  "embeddings": true, "authRequired": true, "version": "0.2.0" }
```

What the key changes: the **Ask** answers and **Create Mission** feature start using the real model, and search upgrades to embeddings. Grading, hints, and scoring stay rule-based on purpose so training is always fair. The key stays on the server (in .env) and never reaches the browser; .env is kept out of git.

10. Backend Files — Explained One by One

Folder: **backend/** (the Python brain). New files in this version are marked **[NEW]**.

main.py

What it is: The front door — starts the server, creates tables, and seeds the demo team.

- Creates tables and seeds the demo team/content on first run (lifespan startup).
- Wires in all six routers and turns on CORS for the frontend.
- Adds public **/api/health** (also reports liveAi/embeddings) and **/**.

app/config.py

What it is: Reads all settings from .env (AI keys, JWT secret, database, flags).

- Exposes **provider_configured** (mock vs live) and **embeddings_configured** (TF-IDF vs vectors).
- Defaults make it run offline: SQLite path, a dev JWT secret, SEED_DEMO=true.

app/database.py [NEW]

What it is: Sets up the database connection (SQLModel/SQLite).

- **init_db()** creates all tables; **get_session()** gives each request a DB session.
- Switching to Postgres is just a DATABASE_URL change.

app/db_models.py [NEW]

What it is: The database tables (User, Team, Membership, Document, Chunk, Mission, MissionAttempt).

- Each content row carries a **team_id** so the API can enforce team isolation.
- Missions and attempts store rich JSON payloads to reuse the existing shapes.

app/security.py [NEW]

What it is: Login helpers: password hashing, JWT tokens, and the request 'gates'.

- **hash_password/verify_password** (PBKDF2) and **create_access_token** (JWT).
- **get_current_user**, **get_team_context**, and **require_admin** protect the routes.

app/seed.py [NEW]

What it is: First-run demo data so the app is usable instantly.

- Creates the demo team, demo admin + member, sample documents (indexed), and 5 missions.
- Only runs when the database is empty and SEED_DEMO is true.

app/retrieval.py [NEW]

What it is: Per-team document search (the 'R' in RAG).

- **index_document** splits a doc into chunks (with embeddings if available).
- **search** ranks chunks for a query — embeddings when configured, else TF-IDF — always team-scoped.

app/ai_provider.py

What it is: The phone line to a real AI model (OpenAI/Azure/Gemini).

- **chat()** for text, **chat_json()** for structured output, **embed()** for search vectors.
- Does nothing in mock mode; raises on real errors so callers can fall back.

app/ai_service.py

What it is: Decides mock vs live and keeps every answer in one shape.

- Ask + Generate Mission try the real model first, then fall back to the mock on any error.
- **_coerce_mission** forces AI missions into a valid shape (exactly one correct choice per step).

app/mock_ai.py

What it is: The offline brain — clever rules that work with no internet.

- game_master, evaluate (grading), learning_report (private score), answer_question,
- onboarding_plan, find_expert, summarize_document, generate_mission.

app/knowledge.py

What it is: Shared text helpers used by retrieval (tokenizer).

- Provides the word-splitting used by the TF-IDF fallback.

app/content.py

What it is: Loads global reference data (acronyms, experts) from JSON.

- These are shared reference lists, not team-specific.

app/models.py

What it is: The Pydantic 'forms' — the single source of truth for every API shape.

- Requests/responses for auth, teams, documents, missions, insights, plus the sanitized
- mission models (answer key removed before sending to the browser).

app/routers/auth.py [NEW]

What it is: Sign-up, login, demo login, and 'who am I' (the only open routes).

- register creates a user + their own team (as admin); login/demo return a token + teams.

app/routers/teams.py [NEW]

What it is: List my teams, create a team, and (admin) manage members.

- Team isolation is enforced by the security gates before any data is touched.

app/routers/documents.py

What it is: Team knowledge base (browse/summarise for all; add for admins).

- Admin upload/paste persists text to the DB and indexes it into searchable chunks.
- Returns each doc with a chunk count; summarize is available to all members.

app/routers/colleague.py

What it is: Digital Colleague endpoints, all team-scoped.

- ask searches ONLY the current team's documents; onboarding/acronym/expert round it out.

app/routers/missions.py

What it is: Team missions: list, play, grade, and save attempts.

- Sends a sanitized mission (no answer key), grades choices server-side,
- and saves each finished run as a private MissionAttempt.

app/routers/insights.py [NEW]

What it is: Admin-only, anonymized team analytics.

- Aggregates attempts into stats like 'X% got this step right first try' — never names or rankings.

11. Frontend Files — Explained One by One

Folder: **frontend/** (the React side you click). New files marked **[NEW]**.

index.html + src/main.jsx

What it is: The starting point; wraps the app in its providers.

- Sets the light/dark theme before paint; mounts App inside Auth + Toast providers.

src/api.js

What it is: The helper for talking to the backend.

- Automatically attaches your login token and selected team id to every call.
- On a 401 it logs you out; turns errors into readable messages.

src/context/AuthContext.jsx [NEW]

What it is: Holds who you are, your teams, and the active team.

- Provides login, register, demo, logout, and switchTeam; remembers you across refreshes.

src/context/ToastContext.jsx [NEW]

What it is: Little success/error pop-up messages.

- Used across pages to confirm uploads, saves, and to show errors.

src/App.jsx

What it is: The shell: auth gate, sidebar, top bar, and routes.

- If not signed in, shows the Login page; otherwise the full app.
- Top bar has the team switcher, AI-mode indicator, theme toggle, and user menu.
- Navigation and routes are role-aware — admin-only items are hidden and blocked.

src/pages/Login.jsx [NEW]

What it is: Sign in / sign up, plus 'Continue as demo user'.

- Create-account mode makes you the admin of a new team.

src/pages/Dashboard.jsx

What it is: Role-aware home page with a 'How it works' panel.

- Admins see Upload / Create mission / Insights; members see Ask / Onboarding / Missions.

src/pages/Documents.jsx

What it is: Team knowledge base with upload clarity.

- Shows supported types, that uploads are team-shared, chunk counts, and indexing toasts.
- Add/upload is admin-only; everyone can browse and summarise.

src/pages/Colleague.jsx

What it is: The Ask page.

- Prominent, clickable/expandable sources; confidence badge; a clear 'not found' state.

src/pages/Missions.jsx

What it is: Mission list with server-saved progress.

- Completion and grade come from the database now, not local storage.

src/pages/MissionPlay.jsx

What it is: The main game screen.

- Clearly labels the two panels: '1. Make your decision' and '2. Investigate with the AI Game Master'.
- Ends with a private report; scores are noted as private.

src/pages/AdminGenerator.jsx

What it is: Create a new mission (admin only).

- Generates a mission, saves it to your team, and lets you play it immediately.

src/pages/Insights.jsx [NEW]

What it is: Team Insights (admin only).

- Shows only anonymized, aggregated stats and per-step 'first try correct' meters.

src/pages/Help.jsx [NEW]

What it is: In-app 'How it works' for admins and members.

- Explains the flow, roles, privacy, and demo-vs-live mode.

src/pages/Onboarding.jsx / Acronyms.jsx / Experts.jsx

What it is: The remaining Colleague tools.

- Onboarding plan builder, acronym explainer, and topic-based expert finder.

src/components/icons.jsx / ui.jsx

What it is: Hand-drawn SVG icons and reusable UI blocks.

- Badges, page headers, stat cards, empty states, skeletons, spinner — no icon library.

src/index.css + tailwind.config.js

What it is: The design system (light/dark, colours, components).

- CSS variables let the whole app flip between light and dark cleanly.

vite.config.js / package.json

What it is: Frontend build settings and dependencies.

- Vite forwards /api to the backend on port 8000; npm run dev starts the app.

12. Data & Storage Files

- **backend/app/data/missions.json** — the 5 sample missions (with answer keys) used to seed a team.
- **backend/app/data/acronyms.json** — 15 acronyms and meanings (global reference).
- **backend/app/data/experts.json** — 6 sample experts and the topics they own (global reference).
- **backend/app/data/documents/*.md** — 5 sample team documents, indexed for the demo team.
- **backend/storage/dbquest.db** — the live SQLite database (auto-created; holds users, teams, docs, missions, attempts). Delete it to reset everything.
- **backend/.env** — your private settings incl. AI key and JWT secret (kept out of git).

13. How to Run It

Backend (Terminal 1):

```
cd backend
python -m venv .venv
.venv\Scripts\activate      # Windows
# source .venv/bin/activate # macOS / Linux
pip install -r requirements.txt
uvicorn main:app --reload --port 8000
```

Frontend (Terminal 2):

```
cd frontend
npm install
npm run dev      # http://localhost:5173
```

Open **http://localhost:5173** and click **Continue as demo user**. It works immediately offline. To use real AI, follow Section 9.

14. Responsible-AI & Privacy Guardrails

- **Team isolation.** Every request is checked against your team membership; you can only see your team's data.
- **Answer keys stay on the server.** Missions reach the browser without the correct answers; grading is server-side.
- **Scores are private.** Individual mission scores are saved privately; only the player sees their own.
- **Insights are anonymized.** Admins see aggregates ('70% spotted the red flag'), never names or rankings.
- **Grounded answers.** The Colleague answers only from your team's documents and admits when it doesn't know.
- **Safe fallback.** If the live AI or embeddings fail, the app uses the offline mock so it never crashes.

In short: sign in, join a team, admins add knowledge, members ask and train, and admins watch anonymized progress — all offline by default, and one .env file away from real AI.